به نام خدا

طراحی کامپایلر و زبان‌های برنامه‌نویسی — بهار ۱۴۰۵

تمرین شماره ۱

آخرین مهلت تحویل: ۲۳ فروردین ۱۴۰۵

سوال ۱: طراحی توکن‌های زبان

دستیار آموزشی: محمد خوشرو

برای یک زبان محاسباتی ریاضی به اسم AMP (Advanced Math Programming) در حال طراحی توکن برای پشتیبانی از اعداد هستیم. در سیستم نوع گذاری (Type System) این زبان، یک نوع داده جامع (Super Type) به نام number وجود دارد که تمامی مقادیر عددی را در بر می گیرد. (با این حال به ازای هر تایپ عددی یک تایپ (data type) اختصاصی نیز وجود دارد تا دقت محاسبات افزایش پیدا کند). این زبان برای تحقق خواسته‌ی ما تنوع بالایی از لیترال‌های عددی و محاسباتی را بپذیرد:

Integer types

- int : اعداد صحیح
- pos_int : اعداد طبیعی (دارای پسوند pos یا n ،در این تایپ در سطح ماشین کد عدد 2^n به‌جای عدد صفر ذخیره می‌کند.)
- u_int : اعداد حسابی یا نامنفی (دارای پسوند u یا w)
- neg_int : اعداد منفی

Floating-Point types

- float : اعداد اعشاری $32bit$ (دارای پسوند اختیاری f - شامل یک ممیز اعشاری کاراکتر نقطه - داشتن همزمان بخش اعشاری و صحیح اختیاری است)
- double : اعداد اعشاری $64bit$ (مشابه float فقط پسوند آن d هست)
- نماد علمی : به صورت $< base > e < exponential >$ که در آن پایه یک عدد float یا int و توان آن یک int به همراه علامت اختیاری است.

Base-N types

- cbase_int : اعداد صحیح با مبنای دلخواه — custom (عدد به صورت $base\$number$ که $base$ عدد طبیعی بزرگتر از 1 و $number$ یک عدد صحیح است)
- bin : اعداد باینری (دارای پیشوند $0B$ یا $0b$)
- quad : اعداد در مبنای 4 (دارای پیشوند $0Q$ یا $0q$)
- octal : اعداد اکتال (دارای پیشوند $0c$)
- decimal : اعداد در مبنای ده (دارای پسوند m که تمام اعداد صحیح و اعشاری ده‌دهی را شامل می‌شود. این تایپ برای از دست ندادن دقت در اثر خطاهای ذخیره‌سازی اعداد به صورت باینری در نظر گرفته شده است. هر رقم از اعداد آن در $4bit$ از حافظه ذخیره می‌شود.)
- hex : اعداد هگزادسیمال (دارای پیشوند اختیاری $0x$ و $0X$ شامل حروف الفبای انگلیسی کوچک و بزرگ قابل قبول $[a-fA-F]$)
- fhex , dhex : اعداد اعشاری هگزادسیمال (مثل اعداد اعشاری float عادی با این تفاوت که بخش های صحیح و اعشاری به صورت hex هستند دارای پیشوند اختیاری $0x$ یا $0X$ همچنان هم پسوند f اختیاری است)
- نماد علمی هگز : مثل نماد علمی عادی فقط به جای e از p استفاده می شود. توان به صورت عددی صحیح با علامت اختیاری است و پایه‌ی آن یک عدد صحیح یا اعشاری به صورت hex
- az_int : اعداد مبنای 36 (دارای پیشوند $0A$ یا $0a$ شامل تمام حروف الفبای انگلیسی$[a-zA-Z]$ و اعداد)

توجه : لیترال‌های هگز بدون پیشوند، نباید با حروف شروع شوند تا با شناسه‌ها (Identifiers) تداخل پیدا نکنند.

قابلیت‌های بیشتر و نکات نیازمند توجه برای طراحی توکن‌ها

- تحلیل‌گر لغوی ما بر اساس طولانی ترین رشته پذیرفته شده در توکن‌های مختلف و سپس اولویت دادن به تایپ های ساده تر توکن بندی را انجام می دهد. (int ساده ترین نوع هست)
- علائم اعداد (مثبت و منفی) به صورت عملیات های یگان (unary operations) توکن بندی می‌شوند و جزوی از توکن خود لیترال‌ها نیستند (نباید آنها را در عبارات منظم خود ندارید)

برای سهولت در نوشتار می‌توانید برای مجموعه اعداد از $[0-9]$ و برای مجموعه حروف از $[a-z]$ یا $[A-Z]$ استفاده کنید (بسته به نیاز شما ممکن است محدوده این مجموعه ها متفاوت باشد). برای هر یک از Literal های مورد بحث در صورت نیاز توکنی منطبق بر تعریف تایپ آنها را بیابید. اگر فکر می‌کنید که نیازی به تعریف توکن جدید برای Literalهای نیست دلیل خود را بیاورید.

سوال ۲: توکن بندی بدون رفع خطا

دستیار آموزشی: فاطمه بلوک آذری

عبارت bccabbbccabcaacaabbcbc با استفاده از قوانین زیر tokenize شده است و به کلمه READER رسیده‌ایم. با توجه به این کلمه، ترتیب قوانین را مشخص کنید و سپس عبارت aaacabcbccaabcc را با این ترتیب tokenize کنید.

A: ab^*c

D: a^*c

E: $a^*b^*c^*$

R: $(a+\varepsilon)bc^*$

سوال ۳: توکن بندی به‌همراه رفع خطا

دستیار آموزشی: مهراد لیویان

می‌خواهیم در مراحل بعد بر اساس هر کدام از زبان‌های زیر توکن‌بندی انجام بدیم.
الف) ابتدا برای هر کدام از توصیف زبان‌های زیر زبان منظم آن را بنویسید.

a. مجموعه اعداد دودویی که شامل 011 نباشند.

b. مجموعه اعداد دودویی که بر 4 بخش‌پذیر باشند.

c. مجموعه اعداد دودویی که با 1 شروع و با 0 پایان یابند.

ب) رشته 11001011010101001101 را ابتدا بر اساس هر کدام از زبان‌ها جدا توکن‌بندی کنید سپس بر اساس تمام زبان‌ها در کنار هم این کار را انجام دهید. در صورت نیاز برای رفع خطا از روش panic mode استفاده کنید. اولویت زبان‌ها برای توکن‌بندی به ترتیب a , b , c هست.

سوال ۴: تبدیل عبارت منظم به اتوماتای قطعی

دستیار آموزشی: کسری کاشانی

در یک زبان آموزشی ساده به نام MiniIR که برای نوشتن راهنماهای بهینه‌سازی در فاز تحلیل واژگانی یک کامپایلر طراحی شده است، تحلیلگر لغوی باید تنها لکسیم‌های مشخصی را تشخیص دهد. فرض کنید در این زبان، حروف کوچک انگلیسی با نماد c، ارقام دهدهی با نماد d، ارقام ناصفر با نماد n و ارقام دودویی با نماد b نمایش داده می‌شوند. همچنین نماد u برای نمایش underscore در این زبان تعریف شده است. در واقع:

- $c = \{a, b, c, \ldots, z\}$
- $d = \{0, 1, 2, \ldots, 9\}$
- $n = \{1, 2, 3, \ldots, 9\}$
- $b = \{0, 1\}$
- $u = \{_\}$

همچنین یکسری کلمات رزرو شده مربوط به خود زبان یعنی RES، اعداد دودویی یعنی BIN، اعداد ده‌دهی یعنی DEC و شناسه‌ها یعنی ID در گرامر این زبان تعریف شده و توکن‌هایی قابل استفاده هستند:

$$\begin{aligned}
\text{RES} &= \{\text{for}, \text{foreach}, \text{fun}, \text{function}\} \\
\text{BIN} &= \{0B\}\ b\ (\ b\ \mid\ u\ b\ )^* \\
\text{DEC} &= \{0\} \cup (\ n\ d^*\ (\ u\ d^+\ )^*\ ) \\
\text{ID} &= c\ (c \mid d)^*\ (\ u\ (c \mid d)^+\ )^*
\end{aligned}$$

توجه شود که در تحلیل لغوی این زبان، قاعده بیشترین انطباق اعمال می‌شود. و هرگاه دو توکن با طول یکسان قابل قبول باشند، اولویت به ترتیب با RES، سپس BIN، سپس DEC و در نهایت ID است.

الف) برای خانواده لکسیم‌های شامل شناسه‌ها و کلمات رزرو شده، یک DFA با نام D1 رسم کنید که دقیقا زبان $L1 = RES \cup ID$ را بپذیرد. همچنین در حالت‌های پذیرش مشخص کنید هر حالت در پایان ورودی چه توکنی تولید می‌کند.

ب) برای خانواده لکسیم‌های عددی، یک DFA با نام D2 رسم کنید که دقیقا زبان $L2 = BIN \cup DEC$ را بپذیرد.
همچنین در حالت‌های پذیرش مشخص کنید هر حالت در پایان ورودی چه توکنی تولید می‌کند سپس مشخص کنید هر یک از رشته‌های زیر توسط این DFA پذیرفته می‌شود یا رد می‌شود.

0 , 00 , 12_34 , _12, 0B10 , 0B_11 , 0B101_ , 0B10_01

پ) بدون رسم کامل، توضیح دهید چگونه می‌توان این DFAهای رسم شده یعنی D1 و D2 را در یک DFA واحد برای تحلیلگر لغوی این زبان ادغام کرد به طوری که هم قاعده بیشترین انطباق رعایت شود و هم اولویت توکن‌ها به درستی اعمال شود.

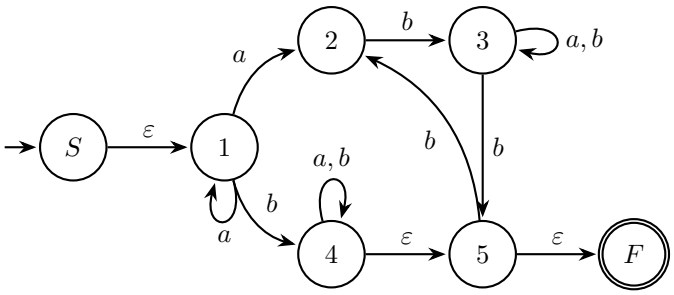
ت) با توجه به بخش قبل، نشان دهید چرا رشته‌ی function2 نباید به صورت توکن‌های FUNCTION و سپس DEC شکسته شود، بلکه باید به صورت یک شناسه واحد یعنی یک ID در نظر گرفته شود. پاسخ باید دقیقا بر پایه تعریف رسمی تحلیلگر لغوی باشد، نه صرفا بر اساس شهود.

سوال ۵: تبدیل اتوماتا به عبارت منظم

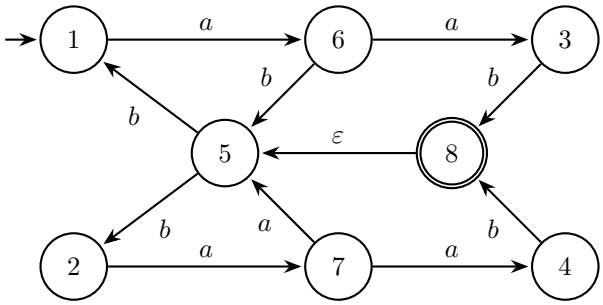
دستیار آموزشی: محمد عرفان دانایی

اتوماتاهای زیر را طبق ترتیب مشخص شده، به عبارت منظم تبدیل کنید.

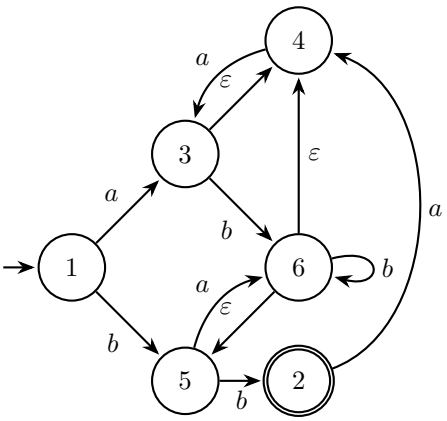
الف)



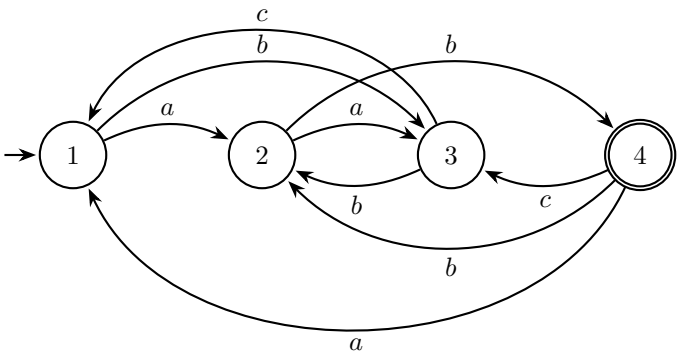
ب)



پ)



ت)



سوال ۶: رسم اتوماتای قطعی

دستیار آموزشی: مهدیس میرزایی

برای هر یک از زبان‌هایی زیر DFA رسم کنید (توجه کنید تعداد استیت‌ها باید کمینه باشد)

الف) برای زبانی که شامل اعداد کوچکتر از 1405 است. $\Sigma = \{0, 1, \ldots, 9\}$

ب) برای زبانی که بین هر دو حرف x تعدادی فردی از سایر حروف الفبا است و حداقل 2 حرف x در هر یک از کلمات این زبان است. $\Sigma = \{a, b, \ldots, z\}$

ج) برای زبانی که زیررشته 021 و 201 ندارد اما حداقل یک زیررشته 210 دارد. $\Sigma = \{0, 1, 2\}$

د) برای زبانی که باقیمانده تعداد x های آن بر سه برابر 0 و باقیمانده تعداد y های آن بر چهار برابر 1 و باقیمانده تعداد z های آن بر پنج برابر 2 است. $\Sigma = \{x, y, z\}$

سوال ۷: نوشتن گرامر ـ امتیازی

دستیار آموزشی: سجاد حسن‌پور

زبان مدل‌سازی Rebeca که توسط اساتید گروه نرم افزار دانشگاه تهران طراحی و توسعه داده شده است بر پایه Actor Model کار می‌کند. برای درک کدها در این سوال، فقط کافی است بدانید که در این زبان هیچ تابعی مستقیماً صدا زده نمی‌شود؛ بلکه Actorها درخواست‌های خود را به شکل یک پیام به Mailbox یکدیگر می‌فرستند تا گیرنده بعداً آن‌ها را به نوبت اجرا کند. برای درک سینتکس این زبان در این سوال، به این ۴ جزء اصلی دقت کنید:

- reactiveclass: معادل class برای تعریف ساختار و رفتار یک Actor
- statevars: متغیرهای محلی و ایزوله‌ی هر Actor (که هیچ Actor دیگری به آن‌ها دسترسی ندارد).
- knownrebecs: لیست Actorهای آشنایی که این موجودیت آدرس آن‌ها را دارد و مجاز است به صندوق پستی آن‌ها پیام بفرستد.

- msgsrv: متدهایی که تعیین می‌کنند وقتی پیام خاصی از صندوق پستی برداشته شد، چه واکنشی نشان داده شود (منطق داخل این متدها مشابه زبان Java نوشته می‌شود).

قطعه کد زیر، یک Actor قطار (Train) را نشان می‌دهد که می‌تواند به یک کنترل‌کننده‌ی پل (bc) پیام بفرستد. با توجه به این کد، به سوالات تحلیل نحوی Syntax Analysis زیر پاسخ دهید:

```
reactiveclass Train(2) {
    knownrebecs {
        BridgeController bc;
    }
    statevars {
        int id;
    }
    msgsrv passed() {
        if (id > 0)
            if (id == 1)
                bc.arrive(id);
        else
            bc.arrive(0);
    }
}
```

الف) یک گرامر مستقل از متن طراحی کنید که بتواند ساختار کلی یک reactiveclass (شامل بخش‌های knownrebecs ، statevars و msgsrv) را دقیقاً مطابق قالب قطعه کد بالا تولید کند توجه کنید که نیازی به نوشتن گرامر برای تمام جزئیات زبان نیست. فرض کنید دستورات داخل msgsrv (که آن را غیرپایانه‌ی Stmt می‌نامیم) فقط می‌توانند شامل دستور شرطی if-else یا ارسال پیام به فرم زیر باشند:

```
id.id(id);
```

ب) با استفاده از گرامری که در بخش الف طراحی کردید، مراحل Leftmost Derivation را برای تولید کل بلوک msgsrv در قطعه کد بالا، گام‌به‌گام بنویسید. (در هر گام از نماد $\Rightarrow$ استفاده کنید تا نشان دهید در هر مرحله سمت چپ‌ترین متغیر جایگزین شده است).

در نسخه‌ی جدید زبان Rebeca، برای مدل‌سازی رفتارهای Recursive در ارسال پیام، از ساختار کنترل جریان handle استفاده می‌شود. هر دستور handle باید در انتها با یک یا دو سیگنال تاییدیه تحت عنوان ack بسته شود تا امنیت حافظه تضمین گردد. دانشجویی برای تولید این ساختار، گرامر زیر را پیشنهاد داده است

$$\text{Stmt} \rightarrow \text{handle Stmt ack} \mid \text{handle Stmt ack ack} \mid \text{skip}$$

ج) نشان دهید این گرامر مبهم است. برای این کار، ابتدا کوتاه‌ترین رشته‌ی ممکن که باعث بروز ابهام می‌شود را پیدا کنید، سپس با رسم ۲ عدد Parse Tree متفاوت برای آن رشته، ادعای خود را ثابت کنید.

د) ابهام در گرامر بالا باعث می‌شود کامپایلر در هنگام بازگشت از توابع تو در تو، نتواند تشخیص دهد که هر تاییدیه (ack) متعلق به کدام سطح از دستورات handle است.

قواعد Stmt را به‌گونه‌ای بازنویسی کنید که ابهام آن کاملاً برطرف شده و دقیقاً همان زبان (مجموعه رشته‌های قبلی) را تولید کند.